

Exercise 1 USB/PS2 Keyboard Interface

Objective

The goal of this exercise is to implement a basic hardware interface for communication with USB/PS2 keyboards, and to examine the various options for implementing VHDL state machines. In addition to this, more experience will be provided in writing behavioral VHDL models and testing them, as well as drawing RTL diagrams. The exercise is split into three parts, with each part focusing on a different implementation of the keyboard interface.

Overview

This exercise focuses on the USB/PS2 keyboard interface, a serial interface for keyboard data. The PS2 protocol, although rarely used nowadays, is still available albeit via USB. The Digilent Nexys4 board includes a USB Keyboard to PS2 Keyboard converter chip that provides the FPGA with a PS2 interface to the keyboard. The PS2 keyboard protocol operates on the basis of scan codes. Whenever a key is pressed on the keyboard, a one-byte *make code* is sent over the PS2 line. Likewise, when a key is released, a two-byte *break code* is sent; this break code typically consists of the hex value “F0” followed by the make code for the key that was released. There are additional codes that are a function of the shift, control, alt, and -lock keys that will not be discussed in this exercise. Figure 1.1 shows a table detailing the various keyboard keys and their associated codes.

ESC 76	F1 05	F2 06	F3 04	F4 0C	F5 03	F6 0B	F7 83	F8 0A	F9 01	F10 09	F11 78	F12 07	
~ 0E	1! 16	2@ 1E	3# 26	4\$ 25	5% 2E	6^ 36	7& 3D	8* 3E	9(46	0) 45	- 4E	=+ 55	Backspace ← 66
TAB 0D	Q 15	W 1D	E 24	R 2D	T 2C	Y 35	U 3C	I 43	O 44	P 4D	[54	] 5B	\ 5D
Caps Lock 58	A 1C	S 1B	D 23	F 2B	G 34	H 33	J 3B	K 42	L 4B	; 4C	' 52	Enter ↵ 5A	
Shift 12	Z 1Z	X 22	C 21	V 2A	B 32	N 31	M 3A	, 41	> 49	? 4A	↑ 59	Shift ↵	
Ctrl 14	Alt 11	Space 29								Alt E0 11	Ctrl E0 14		

Figure 1.1: Source: <https://reference.digilentinc.com/learn/programmable-logic/tutorials/nexys-4-ddr-keyboard-demo/start>

There are two pins on the USB/PS2 interface that are of interest - a data pin and a clock pin. Generally, these pins are bidirectional, but this exercise will only focus on data transmission from keyboard to FPGA. The diagram for a one-byte transmission is shown in Figure 1.2:

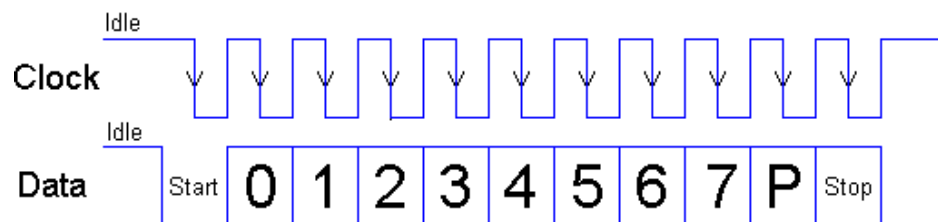


Figure 1.2: Generic format for a one-byte transmission

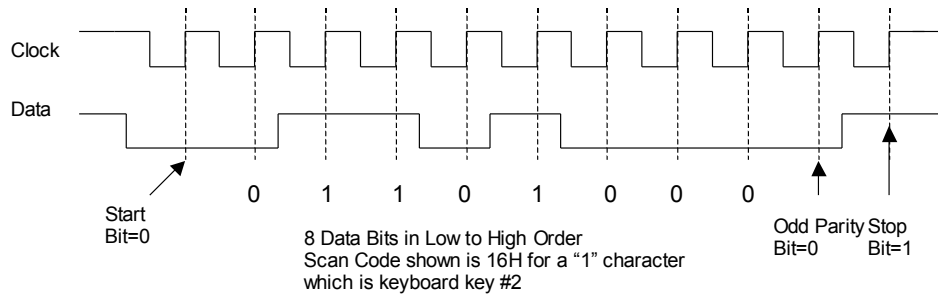


Figure 1.3: Example transmission of the character ‘A’

Transmission begins with a start bit, which is required to be 0, followed by eight data bits, a parity bit (with odd parity) for error-checking, and a stop bit, which is required to be 1. Data can usually be read on either the rising or the falling edge of the PS/2 clock. The clock will only “tick” when data is being sent over the line, otherwise it is idle. The clock period is between 60 and 100 μ s, for a frequency range of 10 to 16.7 kHz.

For this exercise, three separate circuits will be designed, each of which is capable of reading data from the PS/2 and displaying the results on the FPGA’s onboard LEDs. All implementations should also be capable of reading and checking the parity bit for correctness. The LEDs should be capable of displaying the following values: the byte most recently read in from the PS/2 interface, the byte previously read in, the parity bit and calculated parity bit of the most recent byte, and the parity bit and calculated parity bit of the previous byte. The parity bits should be displayed alongside their corresponding byte, and a switch used to select whether that byte is the current or previous one.

The bit file provided in “PS2Interface.bit” may be used to verify that the FPGA and keyboard are working properly. A behavioral implementation is provide in the “PS2_behv.vhd” file. This implementation will be used for the ILA tutorial and is strictly a behavioral implementation; that is, it does not have separate data and control paths. It also must be synthesized and the hardware usage reported, along with the 3 other implementations. “PS2Interface.xdc” provides a sample XDC file. Use section 8 of the Nexys4 Reference Manual to verify that it is correct. There are also pins printed on the board.

Implementation Notes

- For all three implementations, use the **One-Hot** encoding style for the state machine states, and `BTN_UP` as an asynchronous reset. To change to One-Hot encoding, click on the **Project Settings** under **Project Manager** in the Flow Navigator. Under **Synthesis**, change the `fsm_extraction` option to “one_hot.” Click **OK** and then select **No** if it asks about creating another synthesis profile.
- An RTL diagram should be drawn for each implementation, with clear separation between the control unit and the datapath. The control unit is the state machine, and the datapath consists of the rest of the module - any counters, comparators, registers, etc. The state machine can simply be drawn as a box labeled “State Machine” (with labeled inputs and outputs), but it must be accompanied by a full state diagram (A gate-level diagram of the state machine is not required.). In the datapath, remember to include the multiplexer that will be required to select between different LED outputs.

- After drawing the RTL diagram, write VHDL code to implement the circuit for each part. In the VHDL code, there should be clear separation between the control unit and datapath. Remember the correct coding style(s) for VHDL state machines. Each implementation should be tested thoroughly using a testbench.
- After synthesizing each implementation, open the Synthesis Report (the **Vivado Synthesis Report and Utilization Report** option under the **Reports** tab at the bottom) to see the synthesis results. Find the section that details which components were inferred from the design and check whether it matches what was expected from the RTL diagram. The following page shows a sample of the relevant sections, taken from the provided PS/2 interface module. The results will vary.

Vivado Synthesis Report

```
-----
Start RTL Hierarchical Component Statistics
-----

Hierarchical RTL Component report
Module PS2_behv
Detailed RTL Component Info :
+---Adders :
      2 Input      4 Bit      Adders := 1
+---XORs :
      8 Input      1 Bit      XORs := 2
+---Registers :
           16 Bit   Registers := 1
           8 Bit   Registers := 1
           4 Bit   Registers := 1
           2 Bit   Registers := 2
           1 Bit   Registers := 1
+---Muxes :
      4 Input      8 Bit      Muxes := 1
      2 Input      8 Bit      Muxes := 1
      2 Input      4 Bit      Muxes := 1
      5 Input      2 Bit      Muxes := 1
      2 Input      1 Bit      Muxes := 1
      4 Input      1 Bit      Muxes := 2
-----
Finished RTL Hierarchical Component Statistics
-----
```

Utilization Report

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	20	0	63400	0.02
LUT as Logic	20	0	63400	0.02
LUT as Memory	0	0	19000	0.00
Slice Registers	33	0	126800	0.02
Register as Flip Flop	33	0	126800	0.02
Register as Latch	0	0	126800	0.00
F7 Muxes	0	0	31700	0.00
F8 Muxes	0	0	15850	0.00

Part 1: Complex State Machine

Implementation 1 consists of a “complex” state machine and a “simple” datapath. The bit counting is embedded within the structure of the state machine. The circuit should operate in the PS/2 clock domain, meaning that there is no internal system clock used in the FPGA.

Part of this implementation will be given during lecture.

Write VHDL to add a parity-checking circuit to the lecture code. Finally, demonstrate proper functionality by generating a bitstream and loading it on to the FPGA.

Part 2: Simple State Machine

Implementation 2 will consist of a “simple” control unit and a “complex” datapath. The state machine (control unit) should consist of relatively few states, and the datapath should include an explicit counter and comparator, in addition to the parity-checking circuit. As in Part 1, the circuit should operate in the PS/2 clock domain.

Note that “explicit counter” means that the VHDL code should contain a process that implements a counter - structurally defining one and port mapping it is *not* necessary.

Part 3: Introduction to the Integrated Circuit Analyzer (ILA)

Follow the tutorial in “ILATut.pdf” to add an Integrated Logic Analyzer (ILA) element to a new Xilinx project and monitor the FPGA using the Hardware Manager tools. This tutorial will make use of the “PS2.behv.vhd” and “PS2Interface.xdc” files posted to MyCourses. After completing the tutorial, the goal is to configure an ILA to capture an entire break code as shown in Figure 1.4.

There should be two bytes captured - the “F0” byte and the code for the key that was pressed. ILA has several settings that need to be modified to do so:

- The **Trigger Condition Equation** combines one or more Match Conditions to determine when the ILA triggers and starts storing data.
- The **Position** setting specifies how many samples to store before and after the trigger condition occurs.

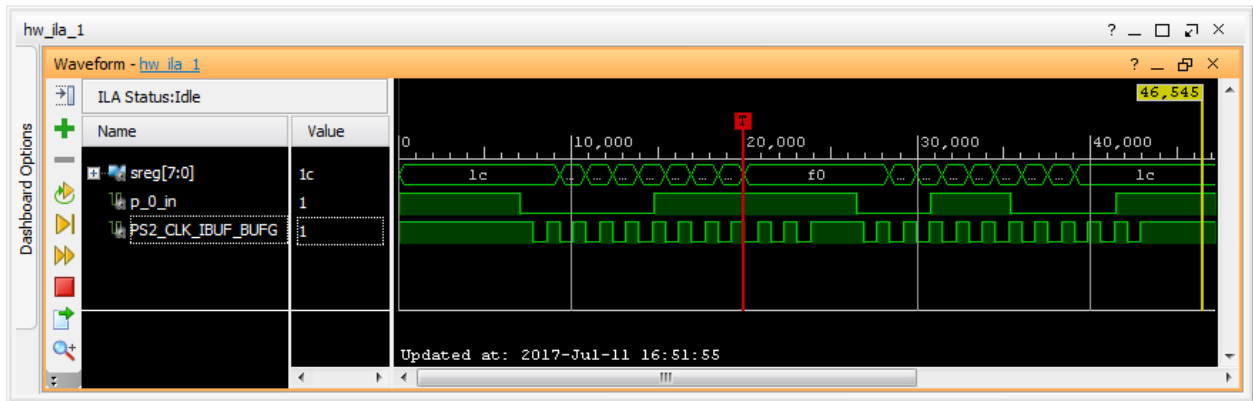


Figure 1.4: Sample ILA capture

Part 4: Separate Clock Domains

Implementation 3 will add synchronization to separate the FPGA clock from the PS/2 clock. Structure the state machine and datapath as in Part 1, but all FPGA components should utilize the FPGA’s 100 MHz clock. The FPGA clock needs to be used together with a clock divider, that will slow it down by 256. See `cnt_clock_divider.vhd` for details on this approach.

Instead of acting as a clock input, the PS/2 clock will now become an enable signal that indicates when the delayed PS/2 data signal needs to be sampled.

Finally, use four serial sampling registers to sample each pin of the PS/2 interface. Sampling in this manner is one of the simplest ways to synchronize two clock domains, reducing the possibility of metastability issues in any flipflops that would need to store the PS/2 clock or data values.

Questions

1. Comment on the synthesis reports for all three implementations plus the behavioral implementation. Include a table of hardware components used and the quantity (adders, subtractors, counters, etc). Discuss whether the quantity of each matches the RTL diagrams.

Parts 1-2 sign-off deadline 9/14/2026

Parts 3-4 and report are due 9/21/2026

Exercise 1: USB/PS2 Keyboard Interface

Student's Name: _____ Section: _____

Demo		Point Value	Points Earned	Date
Part 1: Complex	Hardware Demo	10		
Part 2: Simple	RTL Diagram	10		
	Simulation	10		
	Hardware Demo	10		
Part 3: ILA	Hardware Demo	10		
Part 4: Clock Crossing	RTL Diagram	10		
	Simulation	10		
	Hardware Demo	10		

To receive any grading credit students must earn points for both the demonstration and the report.

Report		Point Value	Points Earned	Comments
Report	Discussion	10		
	Question	10		
Total for demo and report		100		

An electronic submission of project files via MyCourses is required. For information on how to properly submit the files, see the document “MyCoursesTut.pdf” on MyCourses.

Required file structure of tarball submission for this exercise:

```

Ex1
├── Part2
│   ├── [all necessary .vhd files]
│   ├── Part2.xdc
│   └── Part2.tcl
├── Part3
│   ├── [all necessary .vhd files]
│   ├── Part3.xdc
│   └── Part3.tcl
└── Part4
    ├── [all necessary .vhd files]
    ├── Part4.xdc
    └── Part4.tcl

```